

Hardcoded RSA Private Key in Trueview 3MP Smart CCTV Wi-Fi Home Security Camera [T18161-AF]

Affected Product

The Trueview T18161-AF WiFi 3MP Robot Pan-Tilt Security Camera is a smart indoor surveillance camera featuring a 3MP HD sensor, 360° pan-tilt rotation, motion detection, and two-way audio for real-time communication. It supports WiFi connectivity, Alexa integration, and offers remote monitoring through a mobile app.

Firmware Version: 4.9.60.0

Vulnerability Summary

The firmware contains a hardcoded RSA private key used to generate authentication signatures for communications with the cloud API. Because the RSA private key is embedded within the firmware image, any party with access to the firmware can extract the key and generate valid authentication signatures identical to those produced by legitimate devices. The vulnerability affects all devices that share the same embedded private key and undermines the authenticity guarantees of the device-to-cloud communication mechanism.

Vulnerability Discovery

1. Firmware Extraction

The investigation began by extracting the device firmware and unpacking the embedded filesystem to obtain access to the binaries and configuration files contained within the image. The extracted filesystem was examined to understand the components responsible for network communication and cloud connectivity.

2. Initial String Searches

To identify components involved in communication with external services, recursive searches were performed against the extracted files and binaries for network-related artifacts and protocol information. Searches included keywords associated with communication protocols such as TCP, UDP, HTTP, and HTTPS.

Example:

```
grep -rE "tcp|udp|http|https" .
```

```

(kali@rishitha)-[~/Desktop/_truview_firmware.bin.extracted]
$ grep -r -iE "http|https|rtsp|onvif|mqtt|tcp|udp" .
grep: ./43E2C.xz: binary file matches
grep: ./libbusybox.so.1.30.1: binary file matches
grep: ./7CE00C.jffs2: binary file matches
./jffs2-root-0/xsdk.json:      "strServerName":      "RTSP_Server",
./jffs2-root-0/xsdk.json:      "strServerLibPath":    "/usr/lib/libXSDK_Rtsp.so"
./jffs2-root-0/xsdk.json:      "strServerName":      "ONVIF_Server",
./jffs2-root-0/xsdk.json:      "strServerLibPath":    "/usr/lib/libXSDK_Onvif.so"
./services:tcpmux 1/tcp      # TCP port service multiplexer
./services:tcpmux 1/udp
./services:ftp-data 20/tcp
./services:ftp 21/tcp
./services:ssh 22/tcp      # SSH Remote Login Protocol
./services:ssh 22/udp
./services:telnet 23/tcp
./services:nameserver 42/tcp name # IEN 116
./services:syslog 514/udp
./services:#ntp 123/tcp
./services:#ntp 123/udp
grep: ./43D44.xz: binary file matches
./udhcpd.conf:#opt logsrv IP_LIST # 704/UDP log server (not syslog!)
grep: ./squashfs-root-0/lib/libwolfssl.so.18: binary file matches
grep: ./squashfs-root-0/lib/libiconv.so.2.5.1: binary file matches
./squashfs-root-0/local/rsa_private_key.pem:uREG00KJJNmfo50EQQJAQfSA3lR74r/PCIfHfMAUDPoBVTUV9tCcA8kfBEgjGa3
./squashfs-root-0/bin/onvif_config.xml:  <http_max_user13>10</http_max_user13>
./squashfs-root-0/bin/onvif_config.xml:  <https_enable>0</https_enable>
./squashfs-root-0/bin/onvif_config.xml:    <stream_uri>rtsp://192.168.18.201:554/ch0_0.264</stream_uri>
./squashfs-root-0/bin/onvif_config.xml:    <stream_uri>rtsp://192.168.18.201:554/ch0_1.264</stream_uri>

```

During this process, attention was drawn to the `anyka\_ipc` executable, which appeared to contain references associated with cloud communication.

3. Discovery of Cryptographic Artifacts

Additional searches were performed against the `anyka\_ipc` binary. Examination of the strings present within the executable revealed references related to RSA operations and a file named:

`/usr/local/rsa_private_key.pem`

```

(kali@rishitha)-[~/Desktop/_truview_firmware.bin.extracted/squashfs-root-0/local]
$ cat rsa_private_key.pem
-----BEGIN RSA PRIVATE KEY-----
MIICWwIBAAKBgQDDx+10KWatXErm+mKYWCjzX8P2j9Zz5T+ceeo+/ugr1HI2W5mL
ugsURzkFwVdNU+1cYEJiu/oz0OWjUjB3C+uqbwiiv7IDLqil3mpwQ9dObfWs3wsd
92jUCA6KnBIWxNHDSMx2cyNVHEMzmvLB7znj5gGZYXGONJliIYMLUiG42wIDAQAB
AoGANTgv4yB38gKTVqBRTeoMdrkSFWUP52GGI1iHxWP10CoIYK/iZ+mpJuIY2ELk
KPZk30beUfhky9UuL0Vb7E17k8GuGbKmxMVbInguch2vtwL3Jxx9tWy4DBJwX9ji
fH25mS0twZta/BIP96B1AGDw+oNMx74LWDMHlyELPMzR2/ECQQD9ArnLRTA2m1qF
GV0z0VQ7+CSEmW3IAxBbT5/XVmPAUWNSGDoEu+f3bUpBZDdLzrS1e44vbZDtSvL2
ldxCs/mDAkEAXhgZpj0FL2DyJ/NvgzZSFR+QpcaLKpD3VLrS6jHTaarGXyEs7nU
gs3WmE7ZY3dLobWl+1G3K8oPgad3zNYbyQJAFiXr78BhxRX6aZZb00Uce5dw5JA3
j5+9CdFBk88rMkwtpR0bzXhH6R/3FRxZvtgcd5qWSs64AL3D4RQjU0+CUwJAIRqV
7rhdcta4smW0FPFW9Kc4I3vydcKUEoyJ6bLi248muL0eWYqMgO91FS7YF8yTGvW
uREG00KJJNmfo50EQQJAQfSA3lR74r/PCIfHfMAUDPoBVTUV9tCcA8kfBEgjGa3
+i7wEjlljd+RXZKquhFDAs97m3FvXB5ZqeuG0m2W0w=
-----END RSA PRIVATE KEY-----

```

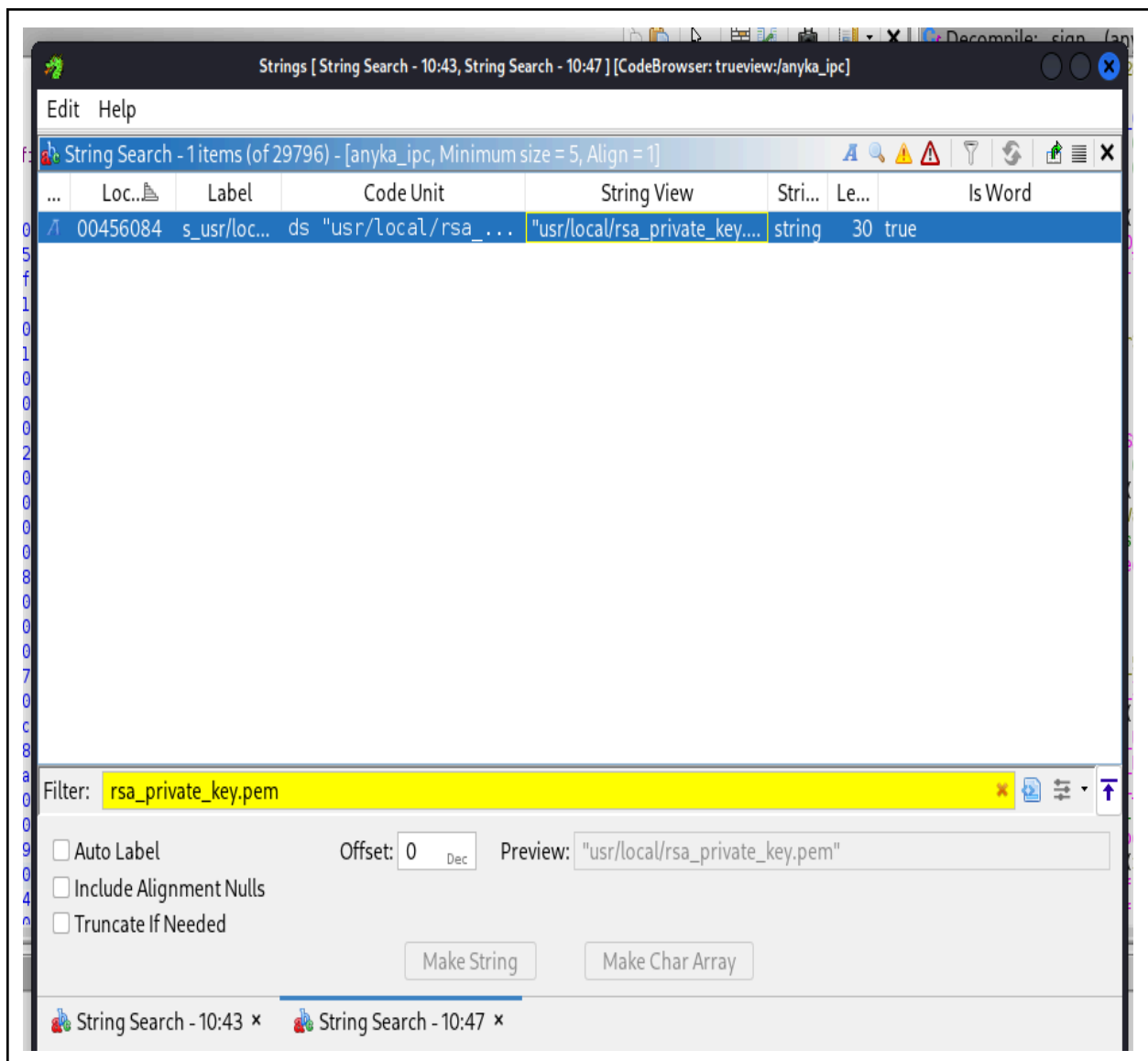
4. Reverse Engineering Using Ghidra

To understand how the key was being used, the `anyka\_ipc` binary was imported into Ghidra for static analysis. Function analysis and cross-reference examination led to the identification of a routine responsible for generating authentication signatures.

Using Ghidra's string search functionality, the string:

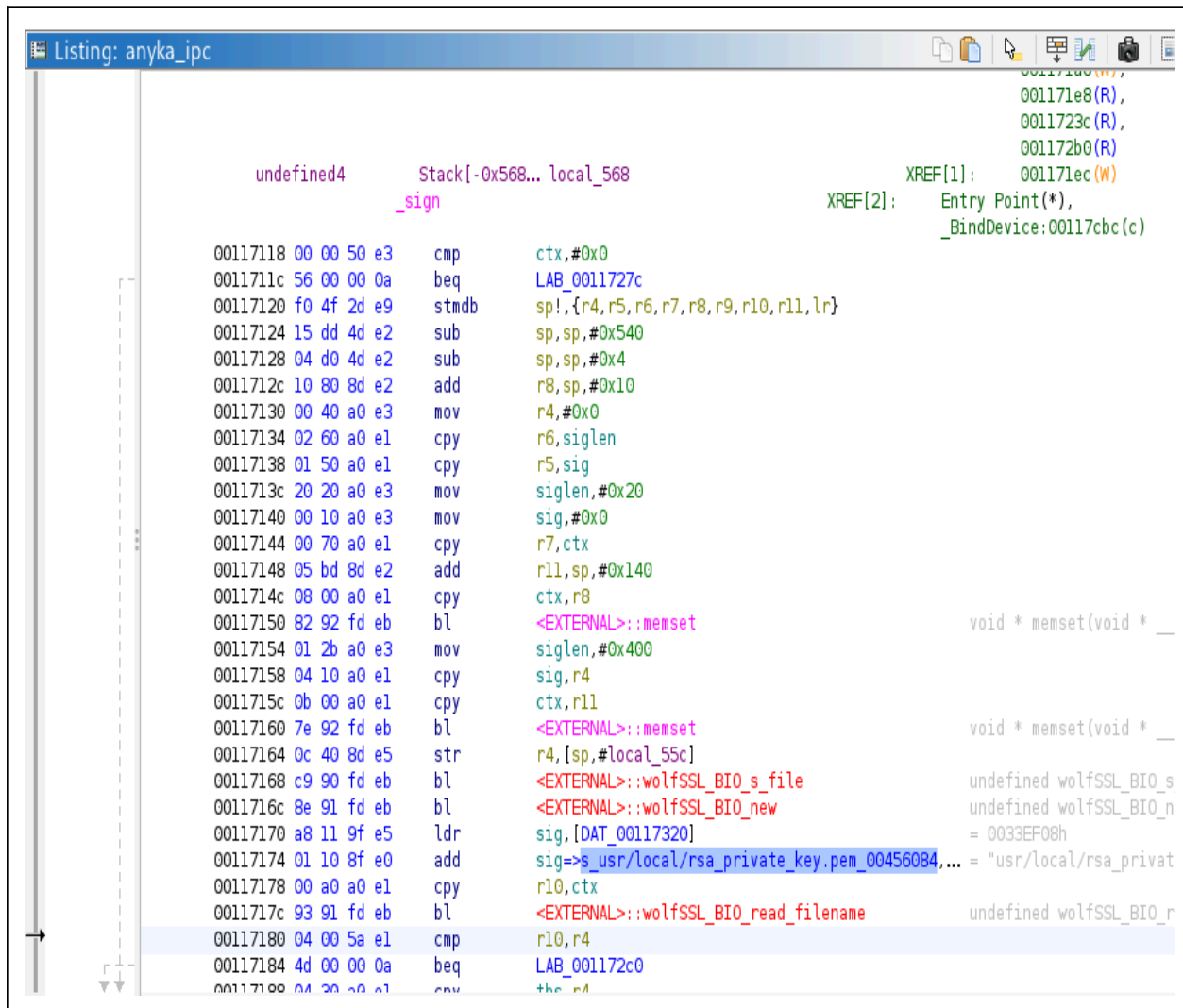
rsa_private_key.pem

was located within the binary. The presence of this filename suggested that the application might contain or reference an RSA private key.



5. Cross-Reference Analysis

Cross-references to the string were examined to determine where and how the key file was used. This analysis led to a function responsible for generating signatures, identified as `_sign`.



```
Listing: anyka_ipc

undefined4      Stack[-0x568... local_568]
_sign

00117118 00 00 50 e3    cmp     ctx,#0x0
0011711c 56 00 00 0a    beq     LAB_0011727c
00117120 f0 4f 2d e9    stmdb   sp!,{r4,r5,r6,r7,r8,r9,r10,r11,lr}
00117124 15 dd 4d e2    sub     sp,sp,#0x540
00117128 04 d0 4d e2    sub     sp,sp,#0x4
0011712c 10 80 8d e2    add     r8,sp,#0x10
00117130 00 40 a0 e3    mov     r4,#0x0
00117134 02 60 a0 e1    cpy     r6,siglen
00117138 01 50 a0 e1    cpy     r5,sig
0011713c 20 20 a0 e3    mov     siglen,#0x20
00117140 00 10 a0 e3    mov     sig,#0x0
00117144 00 70 a0 e1    cpy     r7,ctx
00117148 05 bd 8d e2    add     r11,sp,#0x140
0011714c 08 00 a0 e1    cpy     ctx,r8
00117150 82 92 fd eb    bl      <EXTERNAL>::memset          void * memset(void * __
00117154 01 2b a0 e3    mov     siglen,#0x400
00117158 04 10 a0 e1    cpy     sig,r4
0011715c 0b 00 a0 e1    cpy     ctx,r11
00117160 7e 92 fd eb    bl      <EXTERNAL>::memset          void * memset(void * __
00117164 0c 40 8d e5    str     r4,[sp,#local_55c]
00117168 c9 90 fd eb    bl      <EXTERNAL>::wolfSSL_BIO_s_file  undefined wolfSSL_BIO_s
0011716c 8e 91 fd eb    bl      <EXTERNAL>::wolfSSL_BIO_new    undefined wolfSSL_BIO_n
00117170 a8 11 9f e5    ldr     sig,[DAT_00117320]
00117174 01 10 8f e0    add     sig=>s_usr/local/rsa_private_key.pem_00456084,... = "usr/local/rsa_privat
00117178 00 a0 a0 e1    cpy     r10,ctx
0011717c 93 91 fd eb    bl      <EXTERNAL>::wolfSSL_BIO_read_filename  undefined wolfSSL_BIO_r
00117180 04 00 5a e1    cmp     r10,r4
00117184 4d 00 00 0a    beq     LAB_001172c0
00117188 04 20 a0 e1    cpy     the_r4
```

```
Decompile: _sign - (anyka_ipc)
16 undefined1 auStack_428 [1028];
17
18 if (ctx != (EVP_PKEY_CTX *)0x0) {
19     memset(auStack_558,0,0x20);
20     memset(auStack_428,0,0x400);
21     local_55c = 0;
22     wolfSSL_BIO_s_file();
23     iVar3 = wolfSSL_BIO_new();
24     wolfSSL_BIO_read_filename(iVar3,"usr/local/rsa_private_key.pem");
25     if (iVar3 == 0) {
26         iVar3 = NK_Log();
27         (**(code **))(iVar3 + 0x14))("[%s:%d]open_private_key bio file new error!","_sign",0x14
28         iVar3 = -1;
29     }
30     else {
31         local_55c = wolfSSL_PEM_read_bio_RSAPrivateKey(iVar3,&local_55c,0,0);
32         if (local_55c == 0) {
33             iVar5 = NK_Log();
34             (**(code **))(iVar5 + 0x14))
35             ("[%s:%d]open_private_key failed to PEM_read_bio_RSAPrivateKey!","_sign",0
36             wolfSSL_BIO_free(iVar3);
37             iVar3 = -1;
38         }
39         else {
40             memset(auStack_538,0,0x110);
41             wolfSSL_SHA256_Init(auStack_538);
42             sVar4 = strlen((char *)ctx);
43             wolfSSL_SHA256_Update(auStack_538,ctx,sVar4);
44             wolfSSL_SHA256_Final(auStack_558,auStack_538);
45             iVar5 = wolfSSL_RSA_private_encrypt(0x20,auStack_558,auStack_428,local_55c,0);
46             if (iVar5 == -1) {
47                 iVar5 = NK_Log();
48                 (**(code **))(iVar5 + 0x14))("[%s:%d]RSA_private_encrypt is failed!","_sign",0x157)
49                 wolfSSL_BIO_free(iVar3);
50                 wolfSSL_RSA_free(local_55c);

```

6. Analysis of the Signing Function

The _sign function was analyzed using Ghidra's decompiler. Examination of the decompiled code revealed that the function:

1. Opens the file:

/usr/local/rsa_private_key.pem

2. Loads the RSA private key using WolfSSL APIs:

wolfSSL_BIO_read_filename()
wolfSSL_PEM_read_bio_RSAPrivateKey()

3. Computes a SHA-256 digest of the input data using:

wolfSSL_SHA256_Init()
wolfSSL_SHA256_Update()
wolfSSL_SHA256_Final()

4. Generates an RSA signature using:

wolfSSL_RSA_private_encrypt()

7. Determination of Purpose

Further examination of the surrounding code and associated cloud API routines showed that the generated signature is used for authentication during communications with the cloud backend.

Impact

Successful exploitation allows an attacker to impersonate legitimate devices when communicating with the cloud backend. Possession of the embedded RSA private key enables the generation of valid authentication signatures, causing the server to trust attacker-generated requests.

Because the same private key is embedded in the firmware, extraction of the key from a single device effectively compromises the trust relationship for all affected devices. The vulnerability breaks the authenticity guarantees provided by the RSA-based authentication mechanism and enables device impersonation at scale.

Disclosure Details

Assigned CVE: CVE-2026-51977

Discoverers: B. Sai Sri Vindhya and Rishitha Pucchakayala

Disclosure: This vulnerability was disclosed following responsible vulnerability disclosure practices and has been assigned CVE-2026-51977.